Healthcare Appointment No-Show Prediction Using Machine Learning and Power BI

B.Tech AI & ML Project Report | Student: Aleti Vignesh

1. Introduction

1.1 What Is a Healthcare Appointment No-Show?

A **no-show** happens when a patient books a healthcare appointment but neither attends it nor cancels it in advance. The slot then goes unused, even though a doctor, a room, and staff time were reserved for it.

1.2 Why Missed Appointments Are a Challenge

- **Wasted time and resources:** an unused slot is hard to refill at short notice.
- **Longer waiting times:** other patients may wait longer for an available appointment.
- **Delayed care:** patients who miss visits may face gaps in follow-up and treatment.
- **Scheduling difficulty:** without clear attendance patterns, planning is largely guesswork.

1.3 How Machine Learning Can Help

Machine learning can learn patterns from past appointment records and use them to estimate whether an appointment is more likely to be attended or missed. Such estimates can help staff plan reminders and schedules. They remain estimates, not certainties, about any individual patient.

1.4 Purpose and Scope of the Project

This project analyzes a dataset of **110,521 appointment records**, trains a Decision Tree Classifier to predict appointment attendance, and presents attendance patterns in a Power BI dashboard. The scope covers data cleaning, model training and saving, a prediction script (`predict.py`), and dashboard creation. It is an academic project, and any real-world use would need further validation.

1.5 Role of Power BI

Raw tables and model files are difficult for non-technical users to read. Power BI turns the data into KPI cards and charts, so attendance patterns can be seen, explored, and shared easily.

> **Problem Statement:** Healthcare providers lose time and resources when patients miss appointments without notice. A simple, understandable method is needed to study attendance patterns and estimate which appointments may be missed.

1.6 Project Objectives

- Clean and prepare the appointment dataset for analysis.
- Train a Decision Tree Classifier to predict appointment attendance.
- Save the trained model and generate predictions using `predict.py`.
- Build a Power BI dashboard with four KPI cards and four analytical charts.
- Document the work as a clear, beginner-friendly project report.

2. Abstract

Missed healthcare appointments, commonly called no-shows, waste clinical time and can delay care for other patients. This project, *Healthcare Appointment No-Show Prediction Using Machine Learning and Power BI*, studies appointment attendance patterns and builds a simple prediction workflow to support better scheduling decisions.

The project uses a dataset of **110,521 appointment records**. Using Python and pandas in VS Code, the data was cleaned and preprocessed so that it could be used for machine learning. A Decision Tree Classifier from scikit-learn was then trained to predict whether an appointment is likely to be attended or missed. The trained model was saved with joblib as `outputs/models/no_show_model.joblib`, and a prediction script, `predict.py`, loads the saved model to generate predictions without retraining.

To communicate the results, a Microsoft Power BI dashboard was created. It contains four KPI cards and four analytical charts covering age, gender, SMS reminders, and weekday or appointment date. The dashboard shows that, of the 110,521 appointments, **22,314 were no-shows** and **88,207 were attended**, giving an observed **no-show rate of 20.19%**. These figures are descriptive dataset statistics; they are not measures of model performance. This report does not state accuracy, precision, recall, F1-score, or confusion-matrix values, because formal evaluation results were not part of the verified information available for this project.

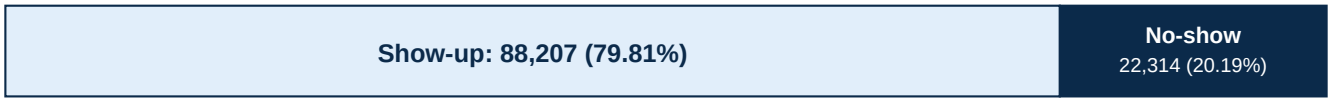
The practical purpose of the project is to show how a complete, beginner-friendly workflow, from raw data to a saved model, a prediction script, and a visual dashboard, can help healthcare staff understand attendance patterns. The patterns in the dashboard are associations in the data; they do not prove that any single factor, such as an SMS reminder, causes a patient to attend or miss an appointment. Predictions are meant to support operational planning and should not replace human judgment. Overall, the project delivers a working end-to-end pipeline and identifies model evaluation, model comparison, and responsible use as important next steps.

Keywords: *healthcare appointments, no-show prediction, Decision Tree, scikit-learn, pandas, joblib, Power BI, data visualization*

Dataset Statistics from the Power BI Dashboard

Measure	Value	Meaning
Total appointments	**110,521**	Number of appointment records in the dataset
No-show appointments	**22,314**	Appointments that were missed by the patient
Show-up appointments	**88,207**	Appointments that were attended
No-show rate	**20.19%**	No-show appointments as a share of all appointments (22,314 out of 110,521)

Share of appointments by attendance outcome (drawn from the dashboard values above)



> **Dataset statistics vs. model performance.** The values above describe the dataset: how many appointments were attended or missed. They do not describe how accurate the Decision Tree model is. No model-evaluation metrics are reported in this document, and none have been estimated or assumed.

3. Tools Used

The project uses a small set of free, widely used tools. The table below lists each tool, what it is used for, how it was used in this project, and why it was a suitable choice.

Tool	Purpose	How It Was Used	Why It Was Suitable
Python	Programming language for the data and prediction workflow.	Used to write the code for loading data, cleaning, training, saving the model, and running `predict.py`.	Easy to read and beginner-friendly, with a large ecosystem of data science libraries.
pandas	Dataset cleaning and manipulation.	Used to load the 110,521 records into tables (DataFrames), inspect them, clean them, and prepare columns for the model.	Built for tabular data, so filtering, cleaning, and transforming records takes only a few clear lines of code.
scikit-learn	Decision Tree classifier training.	Its Decision Tree Classifier was trained on the prepared data to predict whether an appointment is attended or missed.	Offers a consistent, well-documented interface. Decision Trees use simple if/else rules that beginners can follow.
joblib	Saving and loading the trained model.	Used to save the trained model as `outputs/models/no_show_model.joblib` so that `predict.py` can load it.	Avoids retraining every time, and handles saving Python objects such as scikit-learn models efficiently.
Power BI	KPI cards, charts, and an interactive dashboard.	Used to build four KPI cards and four analytical charts: age, gender, SMS reminder, and weekday or appointment date.	Creates interactive visuals without heavy coding, and results are understandable to non-technical audiences.
VS Code	Project development and execution.	Used as the code editor for writing and running the Python scripts, including `predict.py`.	Lightweight, free, and has strong Python support with an integrated terminal.
CSV	Dataset and prediction-result storage, where applicable.	Used, where applicable, to store the appointment data and prediction results in a form that pandas and Power BI can read.	A simple, plain-text table format supported by almost every data tool.

Key Project Files

File	Description
`predict.py`	Prediction script. Loads the saved model and generates predictions.
`outputs/models/no_show_model.joblib`	Saved Decision Tree model created after training.

How the Tools Fit Together

Python and pandas prepare the data, scikit-learn trains the Decision Tree model, and joblib saves it. The script `predict.py` then uses the saved model to generate predictions, and Power BI presents the results as KPI cards and charts. VS Code serves as the workspace for all of the code.

4. Steps Involved in Building the Project

The project was built in ten steps, from collecting the data to documenting the final results. Each step is explained below in simple terms.

1	**Dataset Collection and Inspection.** Load the 110,521 appointment records and examine the columns, data types, and any missing or unusual values.
2	**Data Cleaning and Preprocessing.** Use pandas to fix or remove problem values and make formats consistent. Clean data leads to more reliable results.
3	**Feature Preparation for Machine Learning.** Select the input columns (features) and the attendance outcome (target), and convert them into a numeric form that scikit-learn can use. The exact feature list is defined in the project code.
4	**Training the Decision Tree Classifier.** Train scikit-learn's Decision Tree Classifier so it learns simple rules linking the features to attendance.
5	**Saving the Trained Model.** Save the trained model with joblib as `no_show_model.joblib` so it can be reused without retraining.
6	**Creating and Using predict.py.** Write a script that loads the saved model and generates predictions for appointment records.
7	**Preparing Outputs for Analysis.** Where available, organize actual outcomes, predicted outcomes, and no-show probabilities so they can be analyzed.
8	**Loading the Data into Power BI.** Import the relevant data into Power BI so it can be summarized and visualized.
9	**Creating KPI Cards and Charts.** Build four KPI cards and four charts: age, gender, SMS reminder, and weekday or appointment date.
10	**Reviewing Results and Documenting.** Review the dashboard values, such as the 20.19% no-show rate, and prepare the project documentation.

Project Workflow Diagram

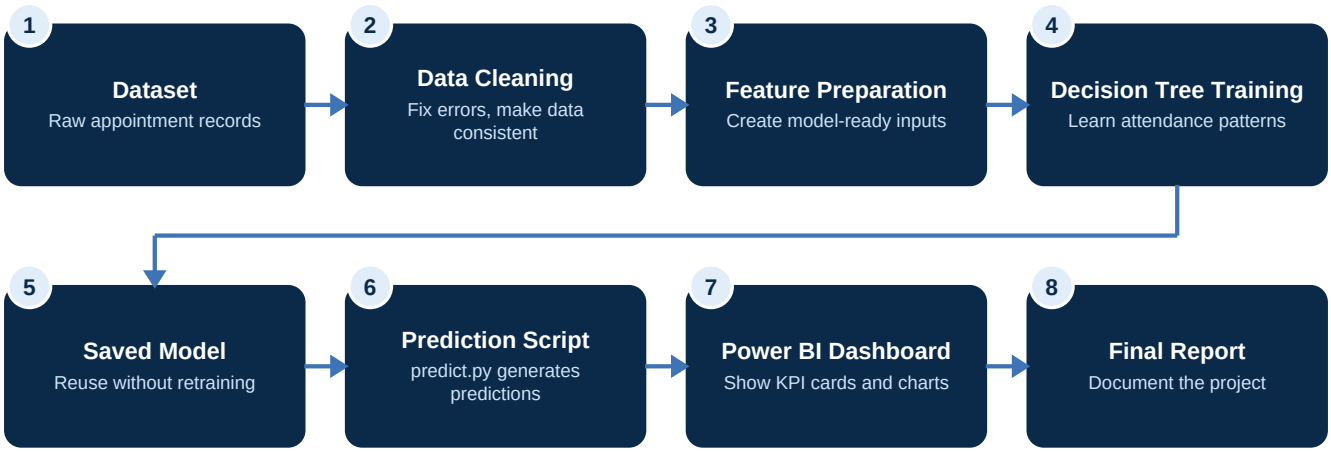


Figure 1: Project workflow from the dataset to the final report. Each box shows the stage and its purpose.

5. Conclusion

What the project achieved. This project built a complete, end-to-end workflow for studying healthcare appointment no-shows. A dataset of 110,521 appointment records was cleaned, a Decision Tree Classifier was trained and saved as `no_show_model.joblib`, a prediction script (`predict.py`) was created, and a Power BI dashboard with four KPI cards and four charts was completed.

Python and machine learning. Python and pandas handled data cleaning and preparation, while scikit-learn's Decision Tree Classifier learned attendance patterns from past records. Saving the model with joblib made it reusable for later predictions.

Power BI and attendance insights. Power BI presented the data as KPI cards and charts on age, gender, SMS reminders, and weekday or appointment date, making attendance patterns easy to read for people without a programming background.

Significance of the 20.19% no-show rate. The dashboard shows 22,314 no-shows out of 110,521 appointments, which is roughly one in five. A share this large matters for scheduling and resource planning. It is a descriptive statistic and does not explain why patients miss appointments; the charts show associations, not causes.

Skills Learned

- Cleaning and preparing real-world data with pandas.
- Training and saving a machine learning model with scikit-learn and joblib.
- Writing a prediction script (`predict.py`) that reuses a saved model.
- Designing KPI cards and charts in Power BI to communicate findings.
- Documenting a technical project clearly.

Potential Improvements and Responsible Use

- **Model evaluation:** measure performance on separate test data using suitable metrics (for example accuracy, precision, recall, F1-score, and a confusion matrix).
- **Model comparison:** compare the Decision Tree with other algorithms before choosing a final model.
- **Responsible use:** protect patient privacy and check that predictions are fair across groups. Predictions should support operational decisions rather than replace human judgment.

Future Scope

- Refresh the dashboard regularly as new appointment data becomes available.
- Carefully test reminder strategies and measure their effect on attendance.
- Package the prediction workflow as a simple application for scheduling staff.

References

[1] pandas documentation. https://pandas.pydata.org/docs/
[2] scikit-learn User Guide: Decision Trees. https://scikit-learn.org/stable/modules/tree.html
[3] joblib documentation. https://joblib.readthedocs.io/
[4] Microsoft Power BI documentation. https://learn.microsoft.com/en-us/power-bi/
[5] Python documentation. https://docs.python.org/3/
[6] Visual Studio Code documentation. https://code.visualstudio.com/docs